



- 請先開啟 VS Code並安裝好環境
- 開啟ChatGPT，建立一個專案Project，名稱為 vibe-coding-weeks，於新增檔案裡上傳今日的講義

正式上課之前

從想法到產品 三週 AI 協作開發之旅

這不是一門教你寫程式的課程，而是一場陪你走完「產品從 0 到 1」完整路徑的實戰旅程。



不是教寫程式 是陪你做產品

我們相信每個人都有好想法，但多數人卡在「不知道怎麼開始」。



Week 1 | 規格化

從想法到可執行規格



Week 2 | 系統化

從能動到像產品



Week 3 | 產品化

從系統到可上線

能力堆疊架構

為什麼零經驗也能成功？

AI 協作與指令能力

學會如何正確地與 AI 溝通，寫出有效的 Prompt

規格化思維

掌握 Schema、Flow、SDD 等規格文件的撰寫

基礎資料能力

理解 CRUD 操作與基本互動邏輯

系統管理能力

狀態管理與 Dashboard 設計

部署與上線能力

雲端部署與環境管理

商業表達能力

產品包裝與 Pitch 技巧

Week 1 建立的能力基礎

核心能力

- 規格化思維（不是寫程式）
- 資料結構意識（Schema）
- 流程設計意識（Flow）
- 與 AI 正確協作的的能力

對應職場能力

- 需求拆解能力
- AI 溝通力
- 產品結構理解力
- 跨領域協作能力

Week 2 建立的能力層次

1 系統思維

2 流程設計能力

3 產品可用性理解

Week 3 建立的能力層次

1 產品落地能力

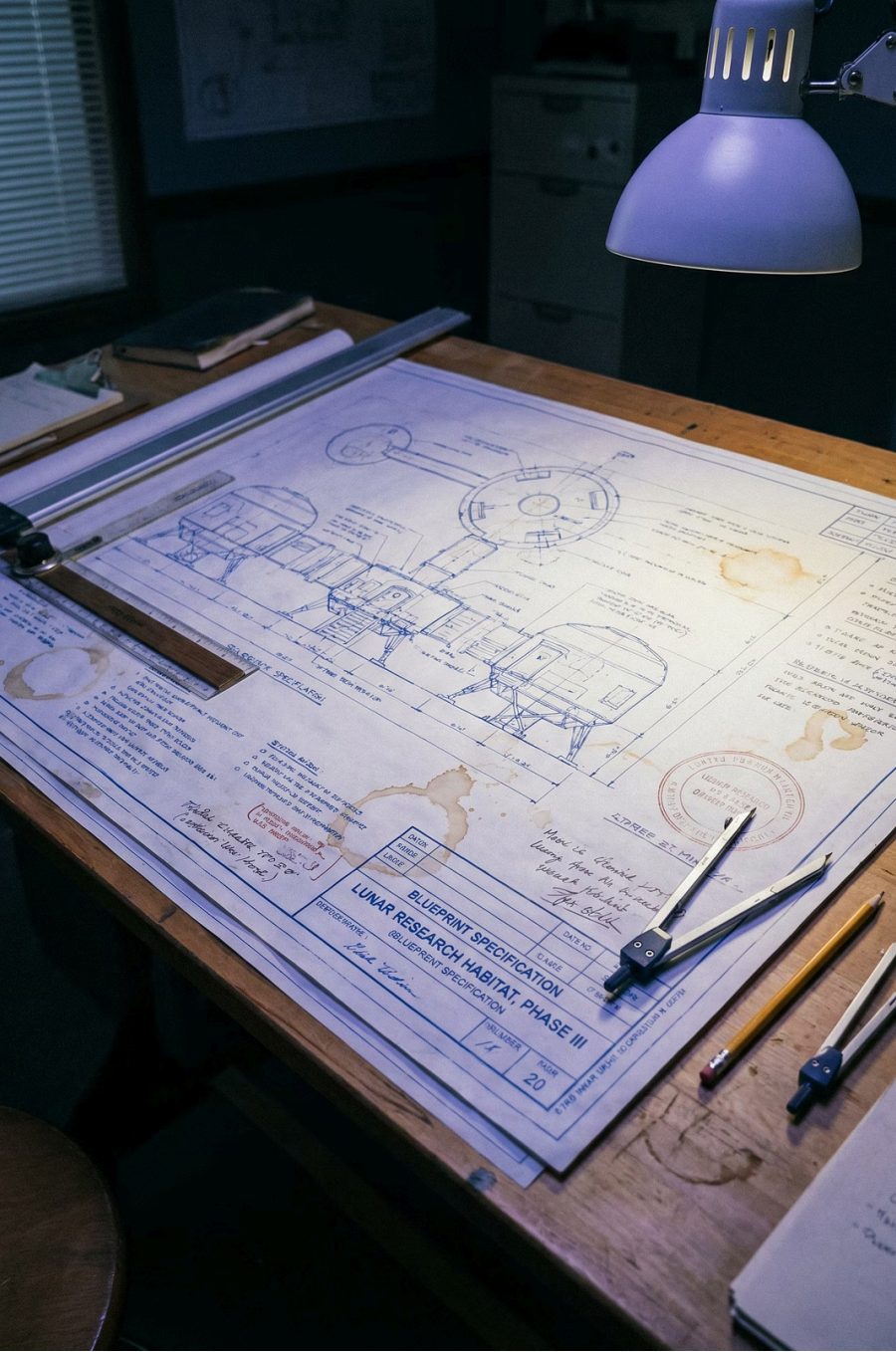
2 跨角色溝通能力

3 商業化思維

Chapter 1

你現在學的不是寫程式 是「把想法變成規格」

在正式開始之前，我們需要先校正一個非常重要的認知。這堂課的核心目標，不是教你成為程式設計師，而是教你如何成為優秀的產品規劃者。



Vibe Coding 失敗的典型路徑

步驟 1

打開 ChatGPT 或其他 AI 工具

步驟 2

說一句話：「幫我做一個 XXX 系統」

步驟 3

AI 生出一大包程式碼

步驟 4

看起來很厲害，但不知道從哪裡開始看

步驟 5

想改一點東西 → 整包壞掉

步驟 6

開始懷疑人生，最後放棄

你（人）的職責

想清楚要做什麼、怎麼做、做到哪裡。定義清晰的邊界與目標，提供明確的規格與約束。

AI 的職責

把你說清楚的東西，轉成程式碼。處理技術細節，生成可執行的實作。

PRD / SDD / DDD 文件對照表

文件	全名	解決的問題	在本課程的定位
PRD 產品層	Product Requirement Document	這是什麼產品？為誰做？解決什麼問題？	要會「講得出來」即可，不需要正式文件
SDD 架構層	Specification-Driven Development	功能怎麼做？資料怎麼存？流程怎麼跑？	Week 1 核心，必須掌握
DDD 執行層	Domain-Driven Design	資料與概念要怎麼分區、不混亂	先理解「精神」，不用精通

SDD 的基本結構預覽：

先給你一個感覺，後面會拆得更細

範例 Schema:

- name (姓名)
- email (電子郵件)
- phone (電話)

範例 Flow:

1. 使用者填寫資料
2. 點擊新增按鈕
3. 系統儲存資料
4. 顯示在列表中



你現在應該能回答這三個問題

1 認知校正

我現在不是在學「寫程式」，而是在學「寫規格」。規格清楚，AI 才能正確執行。

2 文件定位

PRD / SDD / DDD 的用途我已經分得出來。PRD 講給人聽，SDD 講給 AI 聽，DDD 是進階架構概念。

3 問題根源

我知道為什麼「直接叫 AI 做產品」一定會出事。因為沒有明確規格，AI 只能猜測，猜測必然偏離需求。

① 如果你能清楚回答這三個問題，代表這一章的核心概念你已經掌握了。

Chapter 2

為什麼第一週一定要先寫 Spec ?

很多人會想：「我能不能先做出一點東西，再來補規格？」答案是：技術上可以，但實務上你不會補，而且越晚補成本越高。

這一章會告訴你，為什麼 Spec 必須在任何程式碼之前完成，以及一份好的 Spec 應該包含什麼。



先誠實面對：你腦中現在其實是亂的

如果我問你：「你想做的產品是什麼？」

你很可能會這樣回答：

- 一個可以幫我管理 XXX 的系統
- 一個有點像 Notion / Trello 的東西
- 反正就是一個可以用的工具

情境 A：沒有 Spec

你對 AI 說：「幫我做一個客戶管理系統」

AI 會這樣做：

- 自己決定有哪些欄位
- 自己決定畫面長怎樣
- 自己決定流程順序
- 自己幫你加一堆功能

結果：看起來很多東西，但你不敢刪也不敢改，因為不知道會發生什麼事

情境 B：有 Spec

我有一個客戶管理功能，資料只有 name / email / phone，
流程是：填表 → 新增 → 顯示列表，其他都不要做

AI 的狀態：「好，我知道邊界在哪裡了」

結果：功能很單純，但每一個你都看得懂，而且敢改、敢玩、敢延伸

把模糊的想法 轉成可以一步一步照做的說明

釐清誤解：Spec ≠ 一大本文件

你可能的誤解

「那我是不是要寫一份很正式、很厚的文件？」

答案：完全不是

- 一旦開始寫程式

你會被畫面吸走注意力

- 接著

你會開始想加功能

- 最後

你會忘記原本想解決的問題

Week 1 的 Spec 只包含三個核心區塊

區塊	目的	現階段要求
Schema	定義資料長什麼樣	3-5 個欄位
User Flow	定義順序怎麼跑	5-8 個步驟
Scope	定義做到哪就停	明確寫「不做什麼」

拆掉一個錯誤觀念

等我功能比較多，再來補 Spec

等我迷路更久，再來找地圖



你現在應該能回答這三個問題

1 Spec 的本質

你知道 Spec 是「給 AI 的操作說明書」，不是學術文件，也不是展示專業的工具。

2 Spec 的範圍

你知道 Week 1 的 Spec 不需要很厚，只需要 Schema + Flow + Scope 三個部分。

3 Spec 的時機

你知道不先寫 Spec 就開始寫程式，後面一定會還債，而且還得很痛苦。

④ 如果這三點你都清楚了，就可以進入下一章，開始真的動手整理你的想法。

Chapter 3

像PM一樣整理想法

從「我想做一個 App」

到「我現在先做這一個功能」



先打破一個迷思

不是要「設計一個完整產品」

產品思維的第一個關鍵能力

降維思考

錯誤與正確的思考方式

1 太大，無法直接開發

- 我想做一個專案管理系統
- 我想做一個學習平台
- 我想做一個 CRM

2 可立即執行的小單元

- 新增一筆專案資料，並顯示出來
- 新增一筆學習紀錄，並列出列表
- 新增一個客戶，並看到客戶清單

所有產品的第一個功能

90%其實長得都一樣

從「產品想法」到「第一個功能」

1 用一句話描述你的產品

完成這句話：這是一個幫助_____的人，用來_____的工具。

2 問自己一個殘酷的問題

如果我今天只能做一個功能，哪一個功能「不存在」，這個產品就完全沒有意義？

3 把功能翻譯成行為描述

避免抽象概念，改用具體的「新增 X，並顯示 Y」格式。

4 確認可執行性

檢查你的功能描述是否能被直接轉換為程式邏輯。

Step 1：用一句話描述你的產品（PRD 極簡版）

請你先完成這一句話，不用完美，只要清楚：

這是一個幫助_____的人，用來_____的工具。

範例一

這是一個幫助自由接案者管理客戶的工具。

範例二

這是一個幫助內容創作者追蹤產出進度的工具。

範例三

這是一個幫助學習者記錄學習習慣的工具。

Step 2：問自己一個殘酷但有效的問題

❗ 如果我今天只能做一個功能，哪一個功能「不存在」，這個產品就完全沒有意義？

Step 3：把「功能」翻譯成「行為描述」

請避免這種說法

- 管理客戶
- 追蹤學習狀況
- 處理訂單

改成這種

- 新增一個客戶，並在列表中看到
- 新增一次學習紀錄，顯示紀錄清單
- 新增一筆訂單，並顯示訂單列表

你的第一個功能應該長這樣

請你試著填寫下面這段話（這會成為你的 Spec 開頭）：

“我現在要實作的第一個功能是：使用者可以_____，系統會_____。”

範例 1

使用者可以新增一個客戶，系統會顯示客戶列表

範例 2

使用者可以新增一筆習慣紀錄，系統會顯示歷史紀錄

範例 3

使用者可以新增一個內容任務，系統會顯示任務清單

為什麼我們不急著想第二、第三個功能？

把第一個功能轉成 Spec 草稿

現在我們把剛才的思考，正式收斂成一份 Spec 草稿（文字版）。請你照下面格式填寫：

1 功能名稱

例：新增客戶

2 功能描述

使用者可以新增一筆_____，系統會_____。

3 這個功能「不包含」

請明確寫不做什麼，例如：不含登入、不含編輯、不含刪除



你現在應該能回答這三個問題

如果你現在能做到下面三件事，代表你已經準備好進入真正的 SDD：



選定第一個功能

你知道要從哪裡開始



清楚描述功能

可以用一句話說明白



明確排除範圍

知道這個功能目前不包含什麼

Chapter 4

你的產品資料骨架Schema

不先定義資料，後面一定全亂

1 建立正確認知

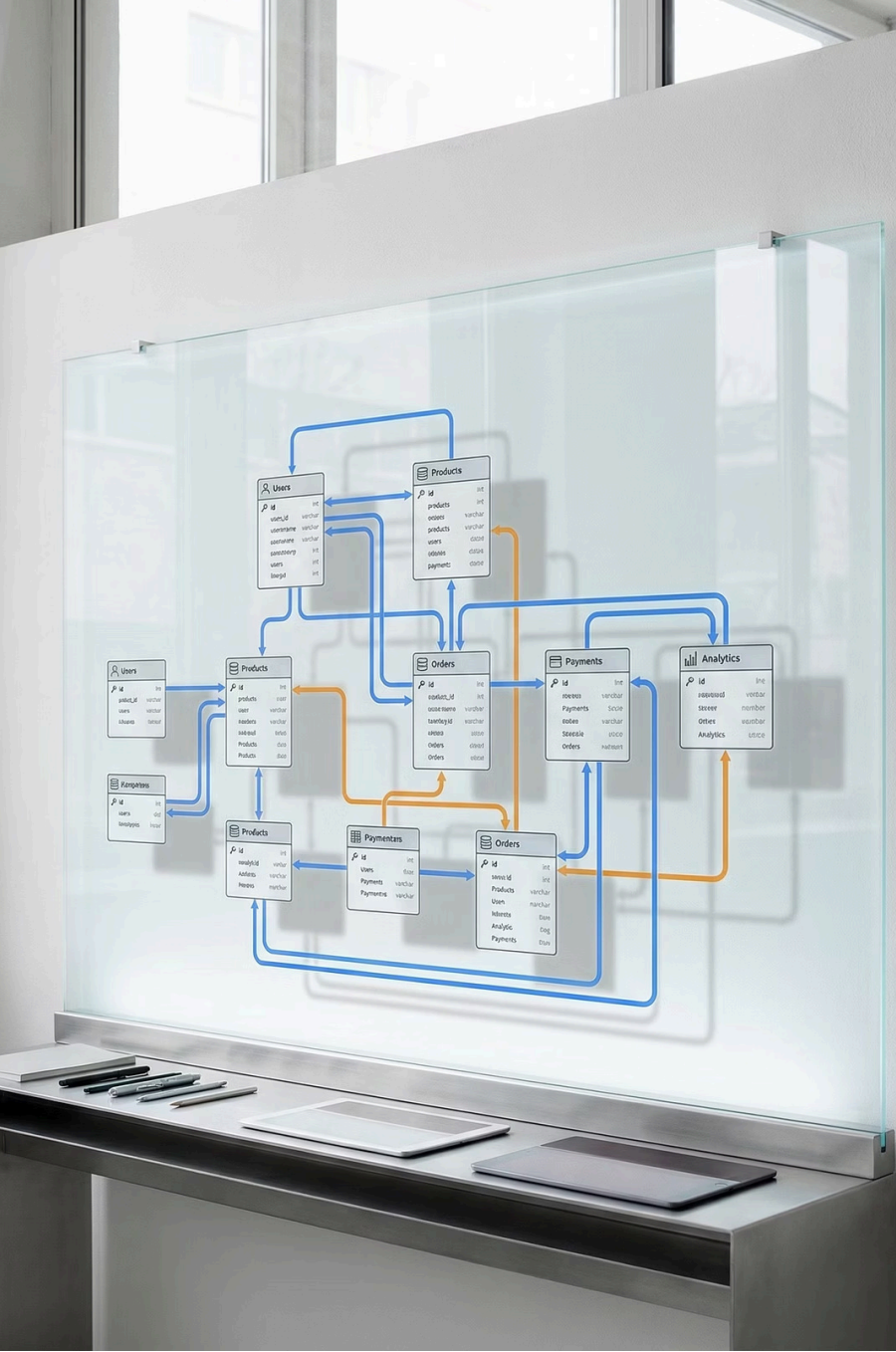
知道你不是在學寫程式

2 選定實作功能

選定了第一個要實作的功能

3 明確範圍邊界

決定了「現在不做什麼」



什麼是 Schema？資料結構定義

Schema 只有一個意思：「這一筆資料，會包含哪些欄位？」

用 Excel 想像

- 每一欄是什麼？
- 每一列代表什麼？
- 資料型態是什麼？
- 哪些欄位必填？

為什麼 Schema 沒定義，AI 一定會亂寫？

因為 AI 在沒有 Schema 的情況下，會做三件災難性的事：

1 自己幫你加欄位

AI 會根據「它認為」你需要的東西，主動新增欄位。

2 用不同名字存同一件事

今天叫 `userName`，明天叫 `user_name`，後天叫 `name`。

3 讓你下一步完全接不起來

當你想要新增第二個功能時，發現前一個功能的資料結構根本接不上，只能整個重寫。

Week 1 的 Schema 請嚴格遵守這四個原則

1 只用 3-5 個欄位

太多會增加複雜度，太少無法運作。

2 欄位名稱要清楚、固定

一旦定義就不要隨意更改。

3 每個欄位都「現在用得到」

不要為了未來先預留。

4 不要為了未來先加欄位

YAGNI 原則：你不會需要它。

範例：第一個功能的 Schema(微型 CRM 客戶管理)

假設你的第一個功能是「新增客戶」，Schema 應該長這樣：

- | | |
|---|---|
|  name 客戶姓名（文字） |  email 客戶信箱（文字） |
|  phone 客戶電話（文字） |  createdAt 建立時間（時間戳記） |

Schema 生成

什麼時候用？

當你已經選好「第一個功能」，但不知道資料欄位該怎麼定義時使用這個指令。

為什麼要這樣下指令？

這個指令的目的不是叫 AI 幫你想產品，而是幫你把「一個功能」翻譯成「資料欄位」。我們透過明確的規則和限制，確保 AI 不會多做也不會少做。

你是一個依照 Specification-Driven Development (SDD) 工作的工程協作 AI。

我現在要實作的第一個功能是：

「使用者可以_____，系統會_____。」

幫我定義這個功能需要的最小 Schema，規則如下：

- 欄位數量限制在 3-5 個
- 每個欄位請說明用途
- 不要為了未來功能先加欄位
- 不要加入我沒有提到的概念

用清單格式輸出 Schema。

你要學會檢查 AI 的 Schema ！這很重要！

AI 給你 Schema 之後，你要自己問三個問題：

1 這個欄位現在真的用得到嗎？

如果答案是「之後可能用到」，請刪掉

2 有沒有欄位其實是在描述同一件事？

例如 userName 和 name，只需要留一個

3 如果我今天不存這個欄位，功能會壞嗎？

如果答案是「不會壞」，請刪掉

Schema 檢查

什麼時候用？

當你已經有一份 Schema，但不確定是否太多、太亂，需要第二雙眼睛幫你檢視時使用。

為什麼要這樣下指令？

避免 AI 自己「優化」成你看不懂的版本。我們只要它提出問題和建議，不要它自作主張修改。

幫我檢查以下 Schema 是否符合「最小可用原則」：

- 欄位數量是否過多
- 是否有功能重疊的欄位
- 是否有現在用不到的欄位

只提出建議，不要自行修改 Schema。

Schema：
(貼上你的 Schema)



你現在應該能回答這三個問題



**一份 3-5 欄位的
Schema**

精簡但完整的資料結構



清楚每個欄位的用途

知道每個欄位「現在用
來干嘛」



**用 AI 檢查過
Schema**

而不是照單全收

① 完成這三項，你就為接下來的開發打下了穩固的基礎。Schema 定義清楚，後面的一切都會順暢許多。

Chapter 5

讓你的系統「動起來」

User Flow

程式不是魔法，是順序



為什麼資料定義完，還不能直接寫程式？

到目前為止，你已經完成一件很多新手沒做的事：

你已經有的

- 知道資料有哪些欄位
- 知道每一筆資料代表什麼
- 有一份清楚的 Schema

什麼是 User Flow ？行為的順序



使用者做了什麼

輸入、點擊、選擇



系統做了什麼

處理、儲存、計算



使用者看到什麼

畫面、訊息、結果

沒有 User Flow 的災難現場：為什麼 Vibe Coding 一定失控？

沒有 Flow 的情況

你對 AI 說：「幫我做一個新增客戶的功能」

AI 可能會：

- 幫你生表單
- 幫你寫 API
- 幫你寫資料庫

但你不知道：

- 什麼時候送資料
- 成功後畫面要不要刷新
- 失敗時怎麼處理

結果：功能「好像有」，但你不敢用

有 Flow 的情況

你先寫好：

1. 使用者填寫表單
2. 點擊「新增」
3. 系統驗證資料
4. 系統儲存資料
5. 系統重新讀取列表
6. 使用者看到最新資料

這時候 AI 幾乎不可能亂寫，因為你已經把「順序」定死了。

Week 1 的 User Flow 撰寫原則，請嚴格遵守

- 1 只寫一條主線流程
- 2 不處理例外、不處理錯誤分支
- 3 每一步都要是「可執行的動作」
- 4 不寫抽象詞(例如：管理、處理)

標準範例：微型 CRM 的 User Flow

Create + Read，這是 Week 1 的標準範例，你之後會一直套用這個結構：

- 1 使用者在表單輸入客戶資料
- 2 使用者點擊新增客戶按鈕
- 3 系統讀取表單內容
- 4 系統將資料寫入資料庫
- 5 系統重新讀取客戶資料
- 6 系統更新畫面上的客戶列表

現在，把你的 Flow 寫出來

完整填寫範例：新增一筆客戶資料(Create + Read)

步驟 1	使用者：在表單中輸入客戶姓名、Email、電話
步驟 2	使用者：點擊「新增客戶」按鈕
步驟 3	系統：讀取使用者輸入的表單資料
步驟 4	系統：將資料寫入資料庫
步驟 5	系統：重新讀取所有客戶資料
步驟 6	使用者看到：畫面上的客戶列表出現剛剛新增的客戶資料

為什麼這個範例是正確的 Week 1 Flow?

1 完整閉環

有「使用者 → 系統 → 使用者」的完整閉環

2 對齊目標

完全對齊 Create + Read 的核心功能

3 範圍明確

沒有偷加登入、驗證、編輯等高風險元素

4 可直接實作

可以 100% 對應到程式實作，不會卡關

User Flow 拆解

什麼時候用？

當你知道「想做什麼功能」，但不知道流程順序怎麼排時。

為什麼要這樣下指令？

這個指令的目的是：逼 AI 按照「使用者 → 系統 → 畫面」的順序思考，而不是直接生程式碼，要用「行為邊界」與「功能範圍」約束 AI。

你是一個依照 Specification-Driven Development(SDD)工作的工程協作 AI。

以下是我目前定義的功能與 Schema：

功能：使用者可以_____，系統會_____。

Schema：(貼上你的 Schema)

幫我產出一條「最小可用功能」的 User Flow，規則如下：

- 只包含一個核心功能
- 只寫一條主流程，不包含任何分支
- 步驟數量控制在 5-8 步
- 每一步都必須明確標示是「使用者」或「系統」
- 不處理錯誤、不處理例外、不加入額外功能
- 不包含登入、權限、驗證、編輯或刪除

用條列清單輸出 User Flow。

Flow 寫完後的三項檢查，你一定要自己確認

請你自己對照檢查下列三個問題，只要有一個「有」，請回去修改：

1 有沒有哪一步「跳太快」？

2 有沒有一步其實是在做兩件事？

3 有沒有出現你 Schema 裡沒有的資料？

Flow 檢查：防止 AI 亂補

當你已經有一份 User Flow，但擔心 AI 在理解或回饋時，偷偷幫你加步驟、加功能或腦補情境時使用。

幫我檢查以下 User Flow 是否符合「單一核心功能、主流程清楚」的原則。依序檢查：

1. 是否只包含一條主流程（沒有分支、沒有例外）
2. 是否有跳過中間步驟，或存在未明確寫出的隱含行為
3. 是否有使用到 Schema 中未定義的資料或欄位
4. 是否有加入未被描述的額外功能

只提出問題與改善建議，不要自行重寫或擴充 User Flow。

User Flow：(貼上你的 User Flow)

Schema：(貼上你的 Schema)



你現在應該能回答這三個問題

到這一章結束，你已經有完整的規格三件套：



最小可用 Schema

3-5 個欄位的精簡資料結構



清楚的 User Flow

不亂加戲的流程描述



能約束 AI 的流程指令

讓 AI 照你的規格走

① 這三樣加起來，就是：真正可以拿來 Vibe Coding 的 Spec。你不再是對 AI 說「幫我做一個功能」，而是給它一份完整的規格說明書。

學習紀錄 App：Schema 到 Flow 的完整範例

Schema

- **topic**：學習主題
- **duration**：學習時長(分鐘)
- **note**：學習筆記
- **createdAt**：記錄時間

User Flow

1. 使用者：輸入學習主題、時長、筆記
2. 使用者：點擊「新增紀錄」按鈕
3. 系統：讀取表單資料
4. 系統：將資料寫入資料庫
5. 系統：重新讀取所有學習紀錄
6. 使用者看到：最新的學習紀錄出現在列表中

① 注意 Schema 和 Flow 的完美對應：Flow 中使用的每個欄位，都能在 Schema 中找到定義。

新手最容易犯的錯：Flow 寫得太複雜

錯誤範例

1. 使用者：登入系統
2. 使用者驗證身份
3. 使用者輸入客戶資料並檢查格式
4. 系統判斷資料是否重複，如果重複則顯示錯誤，否則儲存
5. 系統發送通知信
6. 使用者看到成功訊息並可以選擇繼續新增或返回列表

這個 Flow 包含了太多分支、驗證、和額外功能，完全不符合 Week 1 的原則。

正確範例

1. 使用者：在表單輸入客戶姓名、Email、電話
2. 使用者：點擊「新增客戶」按鈕
3. 系統：讀取表單內容
4. 系統：將資料寫入資料庫
5. 系統：重新讀取客戶資料
6. 使用者看到：客戶列表更新

簡潔、清楚、可執行，沒有任何多餘的步驟。

為什麼要這麼嚴格控制範圍？

你可能會想：「為什麼不能一次就把登入、驗證、錯誤處理都加進去？」答案很簡單：

1 穩定的基礎

2 核心功能

3 進階功能

4 優化與擴充

SDD 的核心哲學

先求有再求好

Specification-Driven Development 不是要你一次做完美的產品，而是要你：



1. 定義清楚範圍

知道現在要做什麼，不做什麼



2. 建立最小可用版本

確保核心功能能跑



3. 逐步迭代優化

在穩定基礎上增加功能

Chapter 6

不是聊天，是規格

Prompt Engineering

你不是在拜託 AI，而是在下工程指令。



為什麼以前的指令，AI 都做得很怪？

如果你以前用過這種 Prompt：「幫我寫一個可以新增資料的網站」，那 AI 其實很可憐。

因為它面臨太多未知：不知道你用什麼技術、不知道資料結構、不知道流程順序、不知道做到哪裡要停。

Prompt = 給 AI 的工程規格

1 不是請求

Prompt 不是禮貌性的拜託或詢問。它是明確的指令，就像你在設定系統參數一樣。

2 不是靈感發想

Prompt 不是用來與 AI 腦力激盪的工具。創意發想應該在你的筆記本上完成。

3 是約束條件集合

Prompt 是一組精確的限制條件，定義了 AI 可以做什麼、不能做什麼，以及應該如何執行。

為什麼先學結構化？

在 Vibe Coding 裡，最容易造成技術債的，不是程式碼本身，而是這兩件事：

1 一次叫 AI 做太多事

當你給 AI 太多任務，它會自行決定優先順序與實作方式，結果往往偏離你的預期。

2 沒有告訴 AI 「不要動哪些地方」

AI 很容易「順手」修改你沒有要求它改的部分，導致本來正常的功能突然出問題。

這一章，就是在解決這兩個核心問題。透過結構化的方法，你將學會如何精準控制 AI 的行為範圍。

SDD Prompt 結構：你的固定模板

你只需要學會一種 Prompt 結構，之後全部套用。我們把它叫做：**SDD Prompt 結構**。

構面	功能說明
角色	限定 AI 在什麼位置思考，例如「前端工程協作 AI」或「資料庫設計顧問」
脈絡	提供已完成的 Spec，包括 Schema、User Flow 等背景資訊
任務	明確只做一件事，例如「建立表單 UI」而不是「完成整個系統」
限制	明確寫出「不要做什麼」，例如不要修改其他檔案、不要新增額外功能

AWS Q Developer × GitHub Copilot 的角色分工

AWS Q Developer（主力）

這是你的主要協作夥伴。你會用它來：

- 根據 Spec 生成程式碼
- 重構特定功能模組
- 進行 Debug 診斷

你對它的態度是：專業工程師

GitHub Copilot（副駕）

這是你的輔助工具。你會用它來：

- 補齊小段程式碼
- 自動完成重複性語法
- 小範圍的程式碼優化

你不會拿它來：下達完整需求或複雜任務

建立工程協作角色

什麼時候用？

在你第一次打開專案、準備開始寫任何程式之前。這是最重要的第一步。

為什麼要這樣下指令？

先把 AI 的角色「鎖定」，後面它才不會一直換人格。這個步驟建立了你與 AI 之間的協作規則。

你是一個依照 Specification-Driven Development (SDD) 工作的資深前端工程協作 AI。

你的任務原則如下：

- 嚴格依照我提供的 Schema 與 User Flow 實作
- 不要自行新增功能
- 不要重構未指定的程式碼
- 每次只處理一個明確任務
- 若資訊不足，請先提出問題，不要自行假設

請回覆「已理解角色與工作方式」。

把 Schema + Flow 餵給 AI（但不叫它寫 code）

很多新手會犯一個錯：一給 Schema + Flow，就立刻說「幫我寫完整程式」。請先忍住。

01

先讓 AI「理解」

確保 AI 正確解讀你的需求規格

02

再讓 AI「動手」

等確認理解無誤後才開始生成程式碼

正確的順序能避免大量返工。這個兩步驟流程看似多花時間，實際上能節省數倍的修正成本。

Spec 對齊確認

什麼時候用？

當你已經完成 Schema 及 User Flow，但還沒開始寫程式。

為什麼要這樣下指令？

避免 AI 誤解你的 Spec，導致生成不符預期的程式碼。

以下是我目前的 Spec：

功能描述：使用者可以_____，系統會_____。

Schema：（貼上你的 Schema）

User Flow：（貼上你的 User Flow）

先幫我確認：

1. 是否有不清楚或矛盾的地方
2. 是否有 Flow 用到但 Schema 沒定義的資料
3. 是否有超出 Week 1 範圍的內容

只做分析，不要產生任何程式碼。

為什麼我們要「一個任務一個 Prompt」？

因為 AI 很容易發生這件事：你叫它改 A，它順手把 B、C、D 全部動掉。

這就是 Vibe Coding 技術債的最大來源之一。

問題範例

你只是要修改表單的驗證邏輯，結果 AI 連資料庫連線、API 路由、錯誤處理都一起改了。原本正常的功能突然全部出問題。

單一任務實作 Prompt

什麼時候用？

當你準備開始實作某一個「明確步驟」。例如：建立表單、寫新增資料邏輯、顯示列表畫面。

為什麼要這樣下指令？

把「可變動範圍」縮到最小，避免 AI 順手改掉其他部分。

依照以下 Spec，只完成「一個任務」：

任務：（例：建立一個可輸入客戶資料的表單 UI）

Schema：（貼上 Schema）

User Flow（僅供參考）：（貼上 Flow）

限制條件：

- 只處理上述任務
- 不要新增其他功能
- 不要修改未提及的檔案
- 若需要新增檔案，請先說明原因

請產出對應的程式碼，並簡要說明放在哪個檔案。

Go Live Server：你一定要「邊做邊看」

每完成一個小任務

就 Go Live 看畫面

為什麼？因為畫面是你唯一「不會騙人」的回饋。

→ 如果畫面怪怪的？不是你笨，是流程或資料有問題

→ 如果功能不如預期？代表 Spec 需要調整

→ 如果完全正常？恭喜，你可以進行下一步了



你現在應該能回答這三個問題

1 Prompt 不是聊天

你已經理解 Prompt 是工程規格，不是隨意的對話

2 會用固定結構下工程指令

你掌握了 SDD Prompt 的四個構面：角色、脈絡、任務、限制

3 知道什麼時候該停下來

你學會了先確認、再執行的兩階段流程

4 不再一次叫 AI 做一堆事

你理解了單一任務原則的重要性

📌 這代表你已經正式進入：可控的 Vibe Coding 狀態。

Chapter 7

實作第一個可跑的功能

不是寫完再看，而是邊做邊跑。

現在我們要讓第一個功能真的動起來。



今天的「極簡 CRM」到底包含什麼？

包含的內容

資料	客戶／案件基本欄位（3-5 個）
功能	Create + Read
互動	表單輸入 → 列表更新
技術	前端 + 雲端資料庫
展示	可 Go Live、可 Demo
定位	作品集等級 Demo

刻意不做的內容

- 登入權限（會讓應用直接炸掉）
- 編輯刪除（Flow 複雜度倍增）
- 搜尋排序（UI 邏輯會亂跑）
- 商業邏輯（Week 2 才做）
- ~~完整 CRM 流程~~（不是今天目標）

📌 今天的目標是先做出一個可用的系統。

我們終於要開始真的做東西了

走到這一章，你已經完成了很多人會跳過、但其實最重要的準備：

明確的功能範圍：你知道你現在要做的是「哪一個功能」

完整的資料定義：你已經定義好 Schema

清晰的流程規劃：你也寫好 User Flow

正確的協作方式：你知道該怎麼下「不會害你重寫整包」的 Prompt



什麼叫做「跑起來」？



程式真的能執行

沒有語法錯誤，沒有執行時期錯誤，邏輯流程完整可運行。



你在瀏覽器中看得到結果

畫面正常顯示，使用者操作有反應，資料確實被處理。

這兩件事缺一不可。只有程式碼能跑但看不到結果，或是有畫面但功能不會動，都不算完成。

使用者輸入資料，並成功新增一筆資料

- 編輯功能
- 刪除功能
- 登入系統

- 資料驗證
- 美化 UI
- 複雜邏輯

專案初始化 (VS Code)

在開始寫任何程式之前，請先完成以下檢查清單：



已安裝 VS Code

確保你使用最新穩定版本



已啟用 AWS Q Developer

擴充套件已安裝並正確設定



已啟用 GitHub Copilot

確認授權狀態正常



已安裝 Go Live Server

Live Server 擴充套件已就緒



請先建立一個全新的專案資料夾，例如：`vibe-coding-week1`，然後在 VS Code 打開它。

專案初始化協作指令

什麼時候用？

當你剛建立好專案資料夾，準備請 AI 幫你規劃最基本的檔案結構時。

為什麼要這樣下指令？

避免 AI 一開始就幫你生一堆你看不懂、也用不到的檔案。保持專案結構的簡潔性。

❗ 你應該只會看到非常精簡的結構，這是對的。通常只需要 3-5 個檔案即可。

你目前正在協助我建立 Week 1 的最小可跑專案。

依照以下原則，幫我建議最基本的前端檔案結構：

- 技術為純 HTML / CSS / JavaScript
- 不使用任何框架
- 目標是支援一個「新增資料」功能
- 可以用 Live Server 直接執行

只提供必要的檔案清單與用途說明，不要先產生完整程式碼。

建立輸入介面（表單）

你的第一個畫面，只需要一件事：讓使用者可以輸入資料

1 輸入欄位

對應 Schema 定義的資料欄位

2 標籤文字

清楚標示每個欄位的用途

3 送出按鈕

觸發資料新增的動作

什麼時候用？

當你已經有專案結構，準備開始做第一個「看得到的畫面」。

為什麼要這樣下指令？

避免 AI 一次幫你把「整個系統」都寫出來。

❗ 專注在「功能先行」的原則。美化與優化都是後續步驟。

依照以下 Spec，幫我建立一個「新增資料」的表單 UI。

功能描述：使用者可以新增一筆資料。

Schema：（貼上你的 Schema）

限制條件：

- 只需要輸入欄位與送出按鈕
- 不需要樣式美化
- 不需要資料驗證
- 不要加入任何其他功能

產出對應的 HTML，並說明放在哪個檔案。

用 Go Live Server 驗證畫面

請你立刻做這件事：

1 在 VS Code 中

確保你正在編輯專案檔案

2 進入 index.html

打開你的主要 HTML 檔案

3 點選右下方 Go Live

啟動即時預覽伺服器

你應該會看到瀏覽器出現：

- 一個可以輸入資料的表單
- 沒有錯誤畫面
- 可以正常顯示在瀏覽器

 如果這一步你看到了畫面，恭喜你，你已經成功一半了。

寫入資料的邏輯 (Create Logic)

現在我們要做的是：把使用者輸入的資料，真的「存起來」。在 Week 1，你只需要先做到：

1 能成功寫入資料

資料確實被儲存到資料庫或暫存區

2 不用考慮錯誤處理

先確保正常流程能跑通

3 不用考慮最佳化

效能優化是後續階段的任務

什麼時候用？

當你的表單已經能顯示，準備讓它「真的有作用」。

為什麼要這樣下指令？

避免 AI 幫你順便加一堆你現在還不需要的東西。

 保持專注在單一任務：讓資料能被成功儲存。

請依照以下 Spec，幫我完成「新增資料」的 JavaScript 邏輯。

Schema：(貼上你的 Schema)

User Flow：(貼上你的 User Flow)

限制條件：

- 只處理資料收集與寫入
- 不處理錯誤情況
- 不新增其他功能
- 不重構未提及的程式碼

請說明需要新增或修改哪些檔案，並提供對應程式碼。

再次 Go Live 確認資料真的有被新增

完成後，請你再 Go Live 一次。此時你應該可以做到：

1 輸入資料

在表單中填入測試資料

2 點擊新增

觸發送出動作

3 沒有報錯

Console 中沒有錯誤訊息

4 功能流程是完整的

從輸入到儲存的整個流程順暢

就算你還看不到列表，也沒有關係。能新增成功，本身就是一個完整的里程碑。





你現在應該能回答這三個問題



第一個 UI

一個可以正常顯示、讓使用者輸入資料的介面。雖然簡單，但它是你的起點。



第一段可用的程式邏輯

從表單收集資料、處理資料、儲存資料的完整流程程式碼。



第一個「真的會動」的功能

一個從頭到尾都能跑通的完整功能。這是質的飛躍。

 這代表：你已經成功跨過「我只是在看 AI 表演」，進入「我在指揮 AI 做事」的狀態了！

Debug 新手最常壞在這裡

壞掉不是失敗

是系統在跟你說話求救

如果你現在的狀態是：畫面有出來、但按鈕按了沒反應、或 Console 出現紅字、或你根本不知道現在到底算不算成功...



Debug 的三個核心目標

這一節的目的不是讓你變成 Debug 專家，而是讓你學會三件事：



不要慌

錯誤是正常的，每個開發者每天都會遇到。保持冷靜才能有效解決問題。



知道問題大概落在哪一類

能夠初步判斷是 UI 問題、邏輯問題、還是資料問題。



知道該怎麼請 AI 幫你修

用正確的方式描述問題，讓 AI 精準修復，而不是亂改一通。

Week 1 Debug 的唯一正確心法

一次只檢查一件事

在這一週，請你牢記這三句話：

1 一次只檢查一件事

不要同時改多個地方，你會搞不清楚
是哪個改動造成的影響

2 不要自己亂改，先確認問題在哪

先診斷，再動手。盲目修改只會讓問題更複雜

3 請 AI 修「點」，不要修「面」

明確限定修改範圍，避免 AI 順手改掉其他功能

什麼情況下，你可以說「我這一關過了」？

實作流程總回顧

1 建立工程協作角色

設定 AI 的工作原則與限制

3 專案初始化

建立最精簡的檔案結構

5 Go Live 驗證畫面

確認表單正常顯示

7 再次 Go Live 驗證

確認功能真的能運作

2 Spec 對齊確認

讓 AI 理解你的 Schema 與 Flow

4 建立輸入表單

完成第一個可見的 UI

6 新增資料邏輯

實作資料寫入功能

8 Debug (如需要)

定位問題、局部修正、回溯檢查

Prompt Engineering 核心原則回顧

SDD Prompt 結構

- 角色：限定 AI 的思考位置
- 脈絡：提供完整的背景資訊
- 任務：明確單一的執行目標
- 限制：清楚定義禁止事項

關鍵心法

- Prompt 是工程規格，不是聊天
- 一次只做一件事
- 先確認理解，再開始執行
- 明確限定可變動範圍
- 邊做邊看，即時驗證

從理論到實踐的關鍵轉變



Before

- 看 AI 表演
- 隨意下指令
- 不知道為何失敗



After

- 指揮 AI 做事
- 結構化下指令
- 能定位與修正問題

Week 1 的真正價值

Week 1 看起來只是做了一個簡單的新增功能，但你實際上獲得了：

可控的 Vibe Coding 能力

你知道如何讓 AI 按照你的意圖執行，而不是隨意發揮

結構化的思考方式

你學會了先規劃、再執行、邊做邊驗證的工作流程

問題定位與修正能力

你不再害怕出錯，因為你知道如何系統性地解決問題



避免技術債的核心策略



明確的 Spec

Schema 與 Flow 定義清楚



限制變動範圍

明確告訴 AI 不能動什麼



精準 Debug

定位問題後局部修正



單一任務原則

一次只做一件事



即時驗證

每完成一步就 Go Live 檢查

AI 協作的角色定位

你的角色

- 定義需求與規格
- 規劃系統架構
- 控制執行範圍
- 驗證輸出品質
- 做最終決策

AWS Q Developer

- 根據 Spec 生成程式碼
- 重構特定模組
- 協助 Debug 診斷
- 執行完整任務

GitHub Copilot

- 補齊小段程式碼
- 自動完成語法
- 小範圍優化
- 提供即時建議

指令模板使用時機總整理

指令類型	使用時機
建立工程協作角色	專案開始前，設定 AI 的工作原則（只做一次）
Spec 對齊確認	完成 Schema 與 Flow 後，開始寫程式前
專案初始化	建立新專案資料夾後，規劃檔案結構時
單一任務實作	每次要實作一個明確功能時（最常用）
定位問題	功能壞掉但不知道哪裡有問題時
局部修正	已知問題位置，需要精準修復時
Flow 回溯檢查	修改多次仍有問題，需要回源頭檢查時

成功的 Go Live 檢查清單



畫面正常顯示

所有 UI 元素都能正確呈現，沒有破版或錯位



表單可以輸入

每個輸入欄位都能正常接受使用者輸入



按鈕有反應

點擊送出按鈕後會觸發對應的動作



Console 無錯誤

瀏覽器開發者工具的 Console 沒有紅字錯誤訊息



資料確實寫入

透過適當方式確認資料已被儲存（例如檢查資料庫或重新整理頁面）

Chapter 8

把資料顯示成列表Read 看起來像產品

當你完成資料新增功能後，接下來最關鍵的一步就是讓資料能被看見。



為什麼 Read 這一步這麼重要？

錯誤認知

很多初學者以為資料有存就好，之後再顯示也沒差。但實際上，沒有 Read 功能，你做的還只是「半套系統」。

正確理解

Read 才是你第一次真正確認系統是完整的地方。它證明了資料不只能存，還能被正確讀取和呈現給使用者。

一個可用功能的三大要素

1 資料能被新增

使用者可以透過介面輸入並儲存新資料到系統中。

2 資料能被讀出

系統能從儲存位置正確讀取已存在的資料。

3 使用者能看到結果

資料能以清楚的方式顯示在使用者介面上。

只有當這三件事都能做到時，你才真的做出了一個「可用的功能」。缺少任何一個環節，系統都不完整。

Week 1 的 Read 目標

在這一週，Read 的目標非常單純：把所有已存在的資料，顯示成一個列表。

請記住這個原則：我們要的是「穩定」，不是「完整」。現階段專注於建立一個可靠的基礎，而不是追求功能的豐富度。

不排序

資料按原始順序顯示即可

不分頁

先顯示全部資料

不搜尋

不加入任何篩選功能

不編輯

只專注於顯示資料

不刪除

保持資料完整性

什麼是 Read 的本質

不是畫面，是資料流動

請先建立一個正確的理解：Read 不是把畫面畫出來這麼簡單，而是把「資料 → 畫面」這條線接起來。

這條資料流動的路徑如果沒接好，後面所有功能都會出現問題。穩固的 Read 機制是整個系統可靠運作的基礎。



系統讀取所有資料

從儲存位置取得完整的資料集合



轉換成可顯示的結構

將原始資料處理成適合在畫面上呈現的格式



渲染到畫面上

將處理後的資料實際顯示在使用者介面

建立資料列表容器

什麼時候需要這個步驟？

當你已經完成新增資料功能，但畫面上還沒有地方可以顯示資料時，就需要先建立一個顯示區域。

為什麼要先建立容器？

先把「顯示區域」固定下來，可以避免後面一直改 UI 結構。這是一個預防性的步驟，能讓後續開發更順暢。

依照目前的功能,幫我在畫面中建立一個「資料列表顯示區」。

限制條件:

- 只需要基本的列表結構(例如 `ul / div`)
- 不需要任何樣式
- 不要加入互動功能

說明需要新增或修改哪個 HTML 區塊。

讀取資料的邏輯

使用者體驗

當使用者重新整理頁面時，能不能看到之前新增的資料？這直接影響系統的可用性。

什麼時候使用？

當你已經可以新增資料，但還沒辦法顯示資料列表時。

為什麼要這樣下指令？

避免 AI 把 Read 跟 Create 混在一起寫，導致流程混亂。保持功能的獨立性，才能確保每個部分都穩定可靠。

- ❏ 記住：清晰的指令能得到清晰的程式碼。模糊的需求只會得到混亂的實作。

系統驗證

這是驗證系統「真的有存東西」的最直接方式。如果資料讀不出來，前面的新增功能就沒有意義。

依照以下 Spec, 幫我完成「讀取資料並顯示列表」的邏輯。

Schema:(貼上你的 Schema)

User Flow(Read 相關):

- 系統讀取所有資料
- 系統更新畫面上的資料列表

限制條件:

- 只處理資料讀取與畫面顯示
- 不加入排序、搜尋或其他功能
- 不修改新增資料的邏輯

說明需要新增或修改哪些 JavaScript 程式碼。

新增資料後列表要怎麼更新？

做法 A：重新讀取（建議）

1. 新增成功後
2. 重新呼叫「讀取資料」的函式
3. 更新整個列表

這是 Week 1 最穩定的做法。

做法 B：手動插入（先不要）

自己手動把新資料插進 DOM。

做法 B 在 Week 1 很容易出錯，我們暫時不採用這個方式。等基礎更穩固後再考慮優化。

1 簡單可靠

邏輯清晰，不容易出錯。重新讀取雖然不是最有效率的方式，但在資料量小的時候完全夠用。

2 容易除錯

如果出現問題，很容易找到原因。因為流程單純，不需要追蹤複雜的 DOM 操作。

3 易於維護

未來要修改或擴充功能時，不會因為過早優化而被綁手綁腳。保持簡單是最好的策略。

指令：新增後刷新列表

什麼時候使用？

當你發現「資料有新增，但列表沒有更新」時。這是新手最常遇到的問題之一。

目前我已經完成:新增資料及讀取資料並顯示列表。協助我在「新增成功後」,重新觸發資料讀取流程,以確保畫面列表會更新。

限制條件:

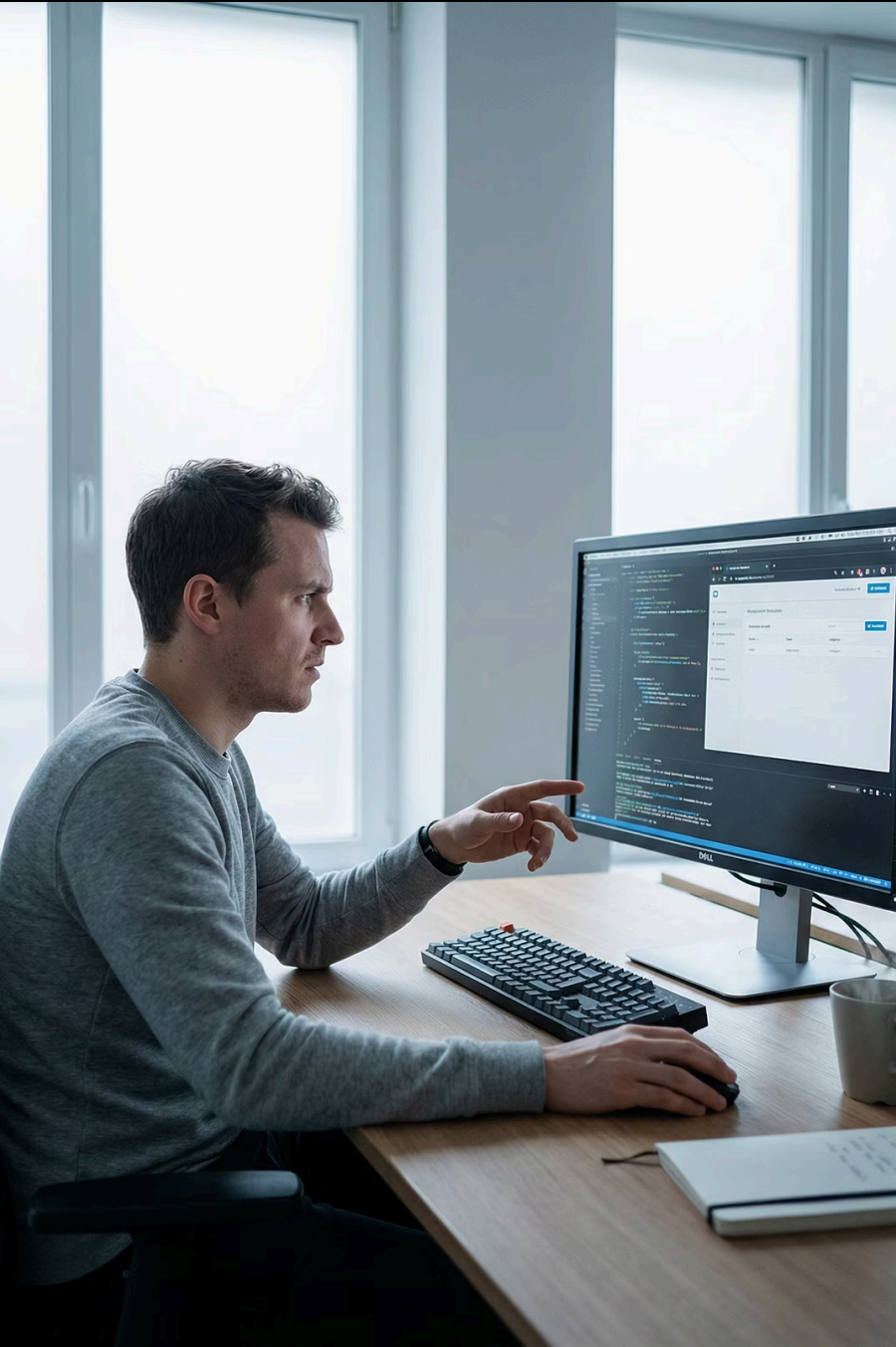
- 不改寫原本的讀取邏輯
- 不新增其他功能

指出需要在哪裡呼叫既有函式即可。

Go Live Server 確認完整流程

現在請你再次使用 Go Live Server，確認以下流程可以完整執行。

這是驗證你的 Read 功能是否正確實作的關鍵步驟。



完整流程檢查清單

1 開啟頁面

確認頁面能正常載入，沒有錯誤訊息

3 輸入資料

在表單中填入新的測試資料

5 列表更新

新資料應該立即出現在列表中

2 看到既有資料列表

之前新增的資料應該自動顯示出來

4 點擊新增

執行新增操作

📌 如果這五步都能跑完：恭喜你，你已經完成了一個完整的 CRUD（Create + Read）循環！



你現在應該能回答這三個問題

請你回頭看一件事：一個小時前，你可能還在想「我真的做得出一個系統嗎？」

但現在，你手上已經有一個真實可用的產品雛形，包含：

- 一個可以新增資料的介面
- 一個會顯示資料的列表
- 一個真的跑得起來的網站

這不是玩具，這是產品雛形。

在進入下一章之前

如果你現在心裡出現這個想法：資料能新增、也能顯示，那是不是代表系統已經完成了？

❏ 答案是：功能完成了，但「產品還沒站穩」。

下一章，我們不會立刻加新功能，而是會先幫你做一件非常重要的事：確認你現在做出來的東西，是不是一個「可以繼續擴充，而不會一路還債」的基礎。

Chapter 9

你現在擁有的是 可擴充的雛形

而不是「還沒完成的半成品」



先幫你正名一件事

如果你現在心裡有一個念頭是：好像還缺很多功能？這樣真的算完成一個系統嗎？

請你先把這個想法放下，因為你現在完成的不是「玩具」，而是：

一個可以穩定運作、可以繼續擴充的產品雛形

這兩者差非常多。前者只是練習，後者是真正的基礎建設。

你刻意沒有做的事情

請注意，下面這些不是你「來不及做」

而是你現在不應該做的事情。這是一個重要的區別

登入/權限

這是另一個完整的系統，需要獨立規劃

編輯/刪除

需要更成熟的資料管理機制

搜尋/排序

基礎功能穩定後再加入

表單驗證

先確保基本流程正確

美化 UI

功能完整後再優化外觀

技術債類型一：功能堆疊太快

常見狀況

- Create 剛跑起來
- 就想加 Edit、Delete、Login
- 覺得功能越多越好

為什麼是債？

- Schema 還沒穩定
- Flow 還沒經過驗證
- 一改就連環出錯

技術債類型二：沒有回頭對齊 Spec

改程式改得很順，覺得很有成就感，但 Spec 文件完全沒更新。

技術債類型三：讓 AI 自由重構

隨口說一句「幫我優化一下」，AI 就幫你重寫了半包程式。

技術債風險檢查指令

什麼時候使用？

當你想在 Week 1 結束後繼續加功能，但不確定是否合適時。

為什麼要這樣下指令？

幫你判斷「這是進步，還是技術債」。讓 AI 扮演顧問角色，而不是直接執行者。

以下是我目前已完成的功能與對應規格(Spec):(貼上你的 Spec)

我正在考慮新增以下功能:(描述你想新增的功能)

只協助我進行「開發風險評估」,並回答以下問題:

1. 這個功能是否適合在目前的系統成熟度下加入?
2. 新增此功能,可能會造成哪些技術債或結構風險?
3. 從開發順序的角度來看,這個功能較適合:
 - 現在就加入
 - 等核心功能更穩定後再加入
 - 等整體架構調整完成後再加入

只提供分析與建議,不要產生任何程式碼或實作方案。

Week 1 正確的進階方向

你現在不需要馬上做這些，但可以先知道正確的發展順序。

這會幫助你在後續做決策時有清楚的判斷標準。

1 Edit / Delete

這是 Schema × Flow 的延伸，需要在 Create/Read 完全穩定後再加入

2 登入系統

這是「另一個完整系統」，需要獨立的規劃和實作

3 UI 美化

應該在所有核心功能都完成後才進行，避免反覆調整

4 商業化功能

需要回到 Spec 重新定義，不能隨意加入

課後六天的三個關鍵問題

在課後六天，如果你想加東西，請先停下來，認真問自己這三個問題：

- 1 這個功能有寫進 Spec 嗎？如果沒有，為什麼要加？
- 2 這個功能是「現在必須」還是「未來可以」？優先順序是什麼？
- 3 加了它，會不會讓原本已經穩定的功能變得不穩定？

只要有一個答案是「不確定」，請先不要做。這不是膽小，這是專業。

這一章你應該帶走的核心觀念

現在不做的不是放棄，是為了可以走更遠

你現在站的位置，是一個非常好的起點。保持這個穩定的基礎，比急著加功能重要得多。

Chapter 10

課後作業與個人產品延伸

這不是交作業，是產品啟動

在 Week 1 結束時，我們不會用「你寫了多少 code」來判斷你有沒有學會。

真正的標準只有一個：

你能不能把同一套流程，套用到「你自己想做的產品」
上



課後作業的真正目的

這份課後作業不是要你做得多完整，而是要確認三件關鍵能力：



獨立撰寫 Spec

你能不能獨立寫出一份
「最小可用規格」



指揮 AI

你能不能用 Spec 指揮
AI，而不是拜託 AI 幫你想



控制範圍

你能不能控制住自己，不
隨意擴充功能

① 只要這三件事成立，Week 1 就是成功的。這比寫出多少程式碼更重要。

作業任務一：定義第一個功能

任務說明

請你用一句話完成下面這段描述：我現在要實作的第一個功能是：使用者可以_____，系統會_____。

重要提醒

- 只能描述一個功能，不能有兩個或更多
- 不要寫成願景或目標，要寫具體行為
- 不要偷偷塞第二個功能進去

這個簡單的描述，會成為你整個專案的核心定位。

作業任務二：完成你的 Spec

你的 Spec 必須包含以下三個區塊，缺一不可：

Schema (3-5 個欄位)

- 欄位名稱與資料型別
- 每個欄位的用途說明
- 不為未來功能預留欄位

User Flow (5-8 步)

1. 使用者：具體操作
2. 使用者：下一步操作
3. 系統：回應動作
4. 系統：資料處理
5. 使用者看到：最終結果

Scope (明確的不做清單)

- 不包含登入功能
- 不包含編輯功能
- 不包含刪除功能

作業任務三：實作 Create + Read

必須完成

- 可以新增一筆資料
- 可以在畫面上看到資料列表
- 可以用 Go Live Server 正常操作
- 資料新增後列表會自動更新

不需要做

- 不需要美化外觀
- 不需要表單驗證
- 不需要效能最佳化
- 不需要錯誤處理

我想將「單一核心功能 → 規格 → 實作」的流程，套用到我自己的產品上。

產品描述：這是一個幫助_____的人，用來_____的工具。

目前我只專注於「第一個核心功能」：使用者可以_____，系統會_____。

協助我完成以下兩件事：

1. 定義一份「最小可用 Schema」
2. 產出對應的 User Flow

遵守以下限制條件：

- 不新增任何額外功能
- 不加入登入、權限、驗證、編輯或刪除
- 不處理錯誤或例外情境
- 不假設未提及的使用情境

先輸出 Schema,再輸出 User Flow。

課後六天建議節奏

我非常建議你用下面這個節奏進行，而不是一口氣狂改。穩定的節奏比衝刺更重要。

Day 1	寫好 Spec（不要寫程式）
Day 2	Schema + Flow 檢查與調整
Day 3	完成 Create 功能
Day 4	完成 Read 功能
Day 5	Debug 與穩定性測試
Day 6	整理成果，準備進入 Week 2

 如果某一天卡住了，這是正常的。請不要硬撐，休息一下或尋求協助。

卡關時的處理原則

不要做的事

- 不要一直盯著螢幕硬想
- 不要隨便改東西試試看
- 不要讓 AI 幫你「重構」
- 不要擴大功能範圍

應該做的事

- 回去檢查 Spec 是否清楚
- 確認每一步的預期行為
- 使用卡關救援指令
- 必要時簡化功能

 卡關救援指令



準備進入 Week 2 的三個條件



穩定的 Create + Read

你的產品有一個穩定可用的新增與顯示功能，經過多次測試都能正常運作。



清楚的資料結構

你知道每一筆資料為什麼存在，每個欄位的用途是什麼，Schema 文件是最新的。



維護的信心

你敢改程式，也知道出問題時怎麼找原因和修復。對系統有掌控感。

如果你符合這三個條件，請放心，你已經準備好進入 Week 2 了。

Week 2 預告：狀態管理

Week 2，我們會在妳已經站穩的基礎上，開始做兩件重要的事情。這會讓你的系統從「能用」變成「好用」。

狀態管理

系統開始變「聰明」，能記住使用者的操作狀態，提供更流暢的互動體驗

Dashboard 設計

開始像真的產品，有完整的資料呈現和導航結構，不再只是單一功能



給自己一些時間

1 不要比較

每個人的學習節奏不同，專注於自己的進步，不要跟別人比較進度。

2 記錄過程

把遇到的問題和解決方法記下來，這會是你最寶貴的學習筆記。

3 享受過程

看到自己寫的程式真的能運作，這種成就感是學習最大的動力。

急著往前的風險

- 基礎不穩固
- 技術債累積
- 挫折感增加
- 失去學習動力

穩步前進的好處

- 每一步都踏實
- 信心持續增長
- 問題容易解決
- 長期持續進步



Spec 先行



專注核心



穩定優先



持續學習



給自己時間

你已經走了多遠

回顧一下 Week 1 開始時，你可能還在想：

- 我真的能學會後端開發嗎？
- CRUD 到底是什麼？
- 怎麼讓資料顯示在畫面上？
- AI 真的能幫我寫程式嗎？

現在，這些問題你都有答案了。而且不只是知道答案，
你還親手做出來了。



接下來的旅程

Week 1 完成

Create + Read 穩定運作

1

2

課後六天

個人產品實作練習

Week 2 開始

狀態管理與 Dashboard

3

4

持續進化

成為產品開發者

恭喜你完成 Week 1

你現在手上有的，是一個真實可用的產品雛形。這不是練習作業，而是可以繼續發展的起點。

好好完成課後作業，我們 Week 2 見！

